



西安电子科技大学
XIDIAN UNIVERSITY

《操作系统》

第2章 进程管理

Process Management

人工智能学院

为什么引入“进程 (Process)”

慌乱的早晨

千头万序

为什么引入“进程（Process）”

■ 待办任务清单（CPU资源严重不足！）

任务	预估耗时	紧急程度	依赖条件
穿衣洗漱	10min	高（形象）	无
整理书包	2min	高（必需品）	穿衣完成
晾衣服	3min	中（不晾会发臭但能忍）	需走到阳台
完成大报告	10min	极高（低分风险）	电脑开机+网络
课堂展示准备	15min	极高（临时通知）	需要PPT和演讲稿
回微信	0.5min	低（但怕错过关键信息）	手机在手

为什么引入“进程 (Process)”

■ 用进程管理的策略解决这个困境

★ 进程 (任务) vs 程序

- 程序：就像你昨晚写好的报告草稿（静态文件）
- 进程：实际“正在写报告”这个动态活动（包含你的打字动作、电脑内存状态等）

★ PCB (Process Control Block, 进程控制块)：你的“任务便利贴”

想象你给每个任务写张小纸条**记录进度**：

- 晾衣服：“已取出3件，还剩5件在洗衣机”
- 写报告：“写到参考文献部分，还差2篇引用“

(操作系统用PCB保存进程状态，方便切换时恢复)

为什么引入“进程 (Process)”

■ 用进程管理的策略解决这个困境

★ 调度策略——你的决策

调度算法	你的对应操作	结果
先来先服务(FCFS)	按任务发现顺序：洗漱→晾衣服→写报告...	报告必然超时！
最短作业优先(SJF)	先做耗时短的任务：回微信→整理书包→晾衣服	课堂展示完全没准备
优先级调度	评估后果：报告DDL > 课堂展示 > 上课签到	可能牺牲晾衣服
时间片轮转(RR)	每件事做2分钟轮流切换	效率极低且精神崩溃 (上下文切换开销)

为什么引入“进程 (Process)”

■ 用进程管理的策略解决这个困境

★ 现实最优解（类比混合调度）

阶段性处理：

- **高优先级任务抢占**：立刻用10分钟写完报告提交

（当你在8:02决定先写报告时，本质上执行了一次进程优先级抢占。就像操作系统强行暂停‘晾衣服’进程，把CPU资源分配给‘写报告’这个高优先级进程。）

- **IO操作并行**：刷牙时把衣服挂起来（异步处理）

（严格来说不属于异步处理，因为刷牙和挂衣服都需要人的肢体参与，并非真正的并行）

- **线程分工**：路上用手机改PPT（课堂展示），同时双腿走路（去教室）

为什么引入“进程 (Process)”

■ 早期的程序：单道环境——顺序执行

现在的程序：多道环境——并发执行

★ 并发程序特征：

- 程序执行的间断性；
- 资源共享；
- 独立和合作(制约)性；

.....

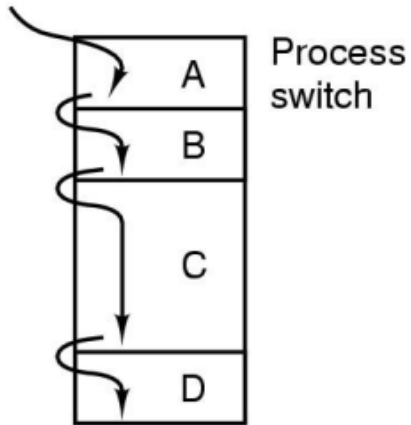


程序的**并发执行**使得程序的执行成为一个**动态性很强**的过程。而**程序**是一个**静态**的概念，不再能切实反映程序执行的各种特征。故而要引入新的概念来表示程序执行过程的这种新特性。

多道程序设计，分时复用技术，共享CPU

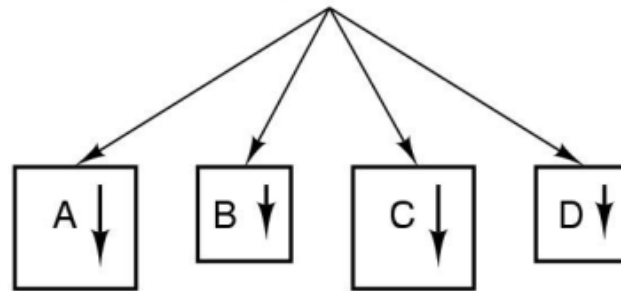
为什么引入“进程 (Process)”

One program counter

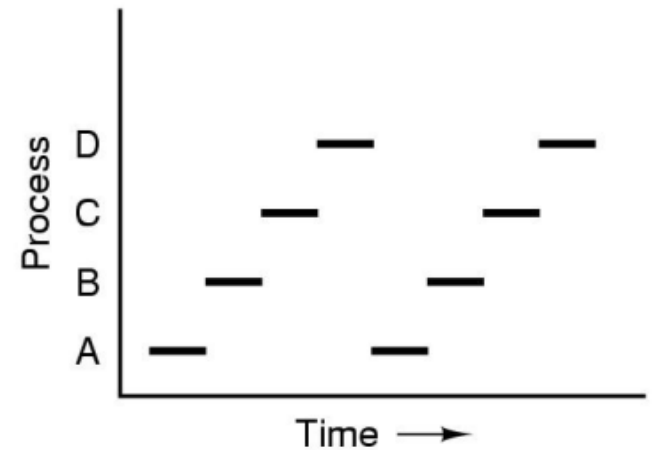


(a)

Four program counters



(b)



(c)

多道程序设计，分时复用技术，共享CPU

★ 多道程序设计 (Multi-programming)

- **思想**：在内存中同时驻留多个程序，当一个程序因I/O操作（如读写磁盘）阻塞时，CPU**立即切换**到另一个程序执行，避免空闲
- **目标**：提高CPU利用率（解决早期单道程序设计的资源浪费问题）
- **特点**：**宏观并行**：用户感觉多个程序“同时”运行；**微观串行**：单核CPU在任何时刻仅执行一个程序

★ 分时复用技术 (Time-Sharing)

- **思想**：将CPU时间划分为**微小时间片**（如10ms），**轮流分配**给多个程序，**快速切换**以模拟“并行”
- **目标**：提升交互性（用户感觉独占计算机，如终端输入即时响应）
- **特点**：**公平性**：每个程序分到均等时间片（防止饥饿）；**快速切换**：依赖硬件时钟中断触发调度

为什么引入“进程 (Process)”

■ 现在的程序：多道环境——并发执行

★ 并发程序三大特征：

• 程序执行的间断性

- 例如：并发程序会因I/O等待、时间片耗尽等被**中断**（如“写报告”时被微信消息打断）
- 思考：程序被中断后**如何恢复**现场？**如何记录**程序当前的执行状态（如变量值、执行到哪一行代码）？

• 资源共享

- 例如：**多个程序共享**CPU、内存、打印机等**资源**（如“写报告”和“听音乐”共用同一台电脑）
- 思考：**如何防止资源冲突**？（如两个程序同时修改同一文件）；**如何公平分配**资源？（避免某个程序饿死其他程序）

• 独立和合作(制约)性

- 例如：程序既需**独立运行**（如浏览器和微信互不干扰），又需**协作**（如下载工具将文件传给播放器）
- 思考：**如何隔离程序错误**（一个崩溃不影响其他程序）？**如何实现程序执行过程中的通信**？

为什么引入 “进程 (Process) ”

■ 进程 (Process) 的作用

- ★ **资源分配单位**：操作系统以进程为单位分配内存、CPU时间等资源
- ★ **并发执行载体**：通过进程切换实现多任务 “同时” 运行（宏观并行，微观串行）
- ★ **隔离与保护**：每个进程有独立的地址空间，防止相互干扰
- ★ **状态管理**：通过进程控制块（Process Control Block, PCB ）保存进程状态（如运行/阻塞/就绪），确保中断后能正确恢复

进程 (Process) 的基本概念

进程(process)的定义

❖ 进程的提出:

- IBM: CTSS/360系统, 称为“工作/任务” (Job/Task)。
- 60年代初, MIT: MULTICS系统首先引入“进程”的概念;

❖ 尚无公认的统一定义。

❖ 直观的定义

- 进程就是进展中的程序 / 执行中的程序

进程 = 程序 + 执行

❖ 教材中:

- 进程是程序的一次执行, 该进程可与其它进程并发执行; 它是一个动态的实体, 在传统的操作系统设计中, 进程既是资源的基本分配单元, (也是基本的执行单元)。

进程 (Process) 的基本概念

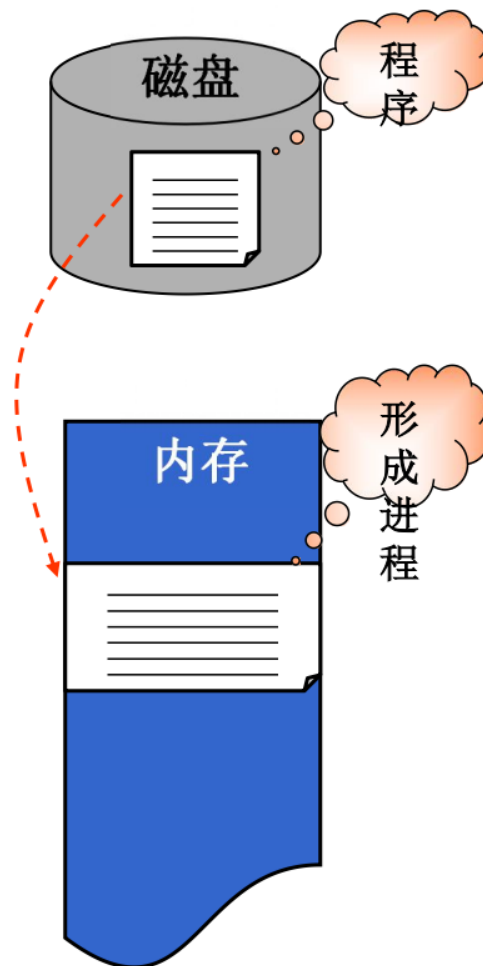
进程与程序的区别和联系

❖ 区别:

- 程序是**静态**的, 是有序代码的集合; 进程是**动态**的, 是程序的一次执行。
- 程序是**永久**的, 没有生命周期, 可长久保存; 进程是**暂时**的, 有生命周期, 是一个动态不断变化的过程。
- 进程是操作系统**资源分配和保护的基本单位**, 程序没有此功能。
- 进程与程序的结构**不同**。

❖ 联系:

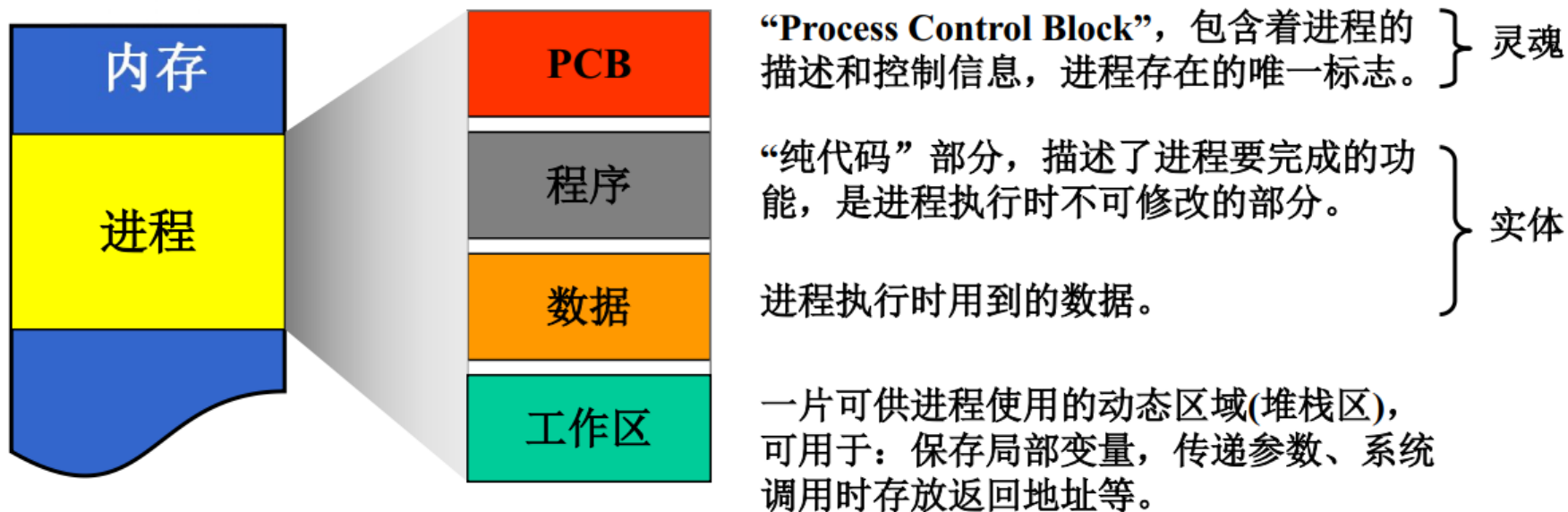
- 通过多次执行, 一个程序可对应多个进程; 通过调用关系, 一个进程可包括多个程序。



进程 (Process) 的基本概念

进程的组成

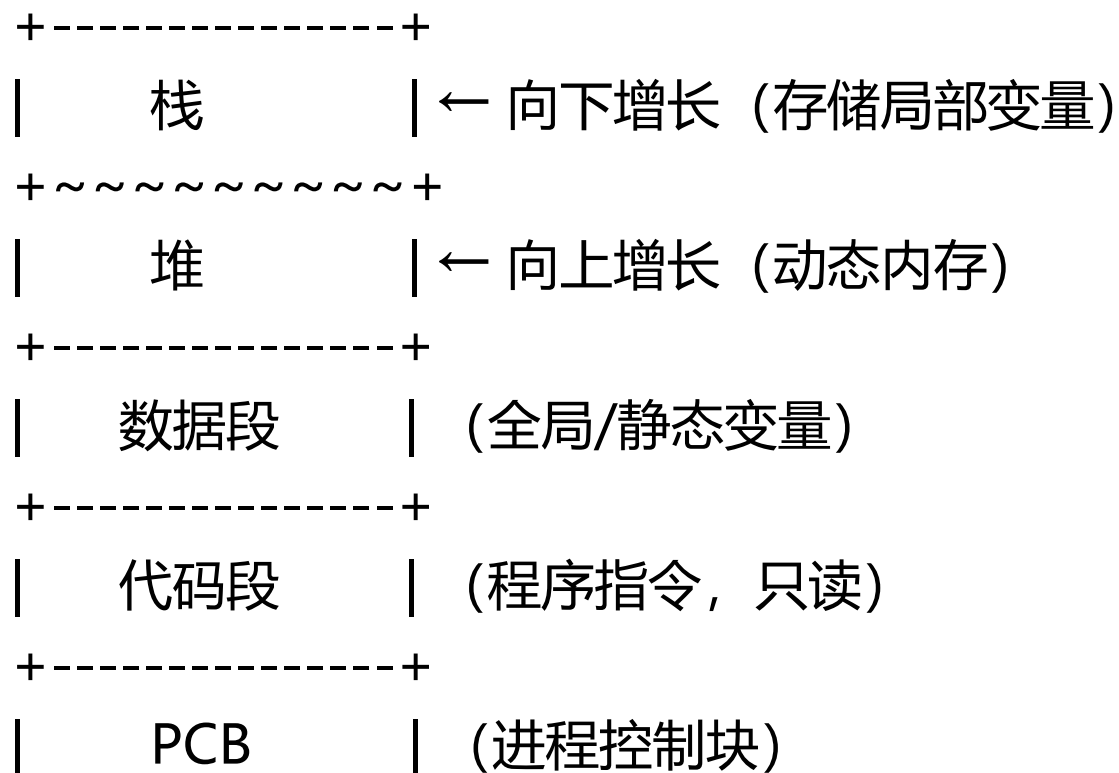
❖ 进程的存储映像



进程 (Process) 的基本概念

★ 进程的内存布局示例

高地址



低地址

进程 (Process) 的基本概念

★ **堆和栈是进程内存中两种动态分配数据的区域**，它们在管理方式、用途和生命周期上有本质区别

特性	栈 (Stack)	堆 (Heap)
管理方式	编译器自动分配/释放 (LIFO)	程序员手动分配/释放 (如 malloc/free)
速度	快 (仅移动栈指针)	慢 (需查找可用内存块)
内存分配方向	向低地址增长 (栈顶在下)	向高地址增长 (堆顶在上)
碎片问题	无 (严格顺序分配)	有 (频繁分配释放易产生碎片)
容量限制	较小 (默认几MB, 可能栈溢出)	较大 (受系统虚拟内存限制)
存储内容	局部变量、函数参数、返回地址	动态分配的数据 (如对象、数组)
线程安全	线程独占 (每个线程有自己的栈)	线程共享 (需同步机制)

进程 (Process) 的基本概念

★ 进程控制块(Process Control Block, PCB)

❖ 定义:

- 是操作系统用来记录进程详细状态和相关信息的基本数据结构，它和进程是一一对应的，是**进程存在的唯一标识**。
- 进程的档案，描述进程的特征，记载进程的历史，决定进程的命运。

❖ 作用:

- 提供进程的各种信息，以便操作系统查询、控制和管理。

◆ PCB与进程生命周期

- **创建进程时:** 操作系统分配PCB并初始化 (如设置初始状态为“就绪”)
- **进程运行时:** PCB动态更新 (如程序计数器PC随指令执行变化)
- **进程切换时:** 保存当前PCB到内存，加载下一个进程的PCB到CPU
- **终止进程时:** 释放PCB占用的内存，从进程表中删除条目

进程 (Process) 的基本概念

★ 进程控制块(Process Control Block, PCB)

PCB结构

类型	内容	作用
标识信息	进程ID, 用户ID, 创建时间, 家族树指针(子进程, 父进程, 孙子进程, 祖父进程...)	标识一个进程
现场信息	CPU通用寄存器内容 CPU状态寄存器内容	记录处理机现场信息, 以备恢复之用
控制信息	持有句柄, 进程状态 信号量, 调度信息 队列指针, 优先级 资源占用信息	用于进程的调度管理

Process management

Registers
Program counter
Program status word
Stack pointer
Process state
Priority
Scheduling parameters
Process ID
Parent process
Process group
Signals
Time when process started
CPU time used
Children's CPU time
Time of next alarm

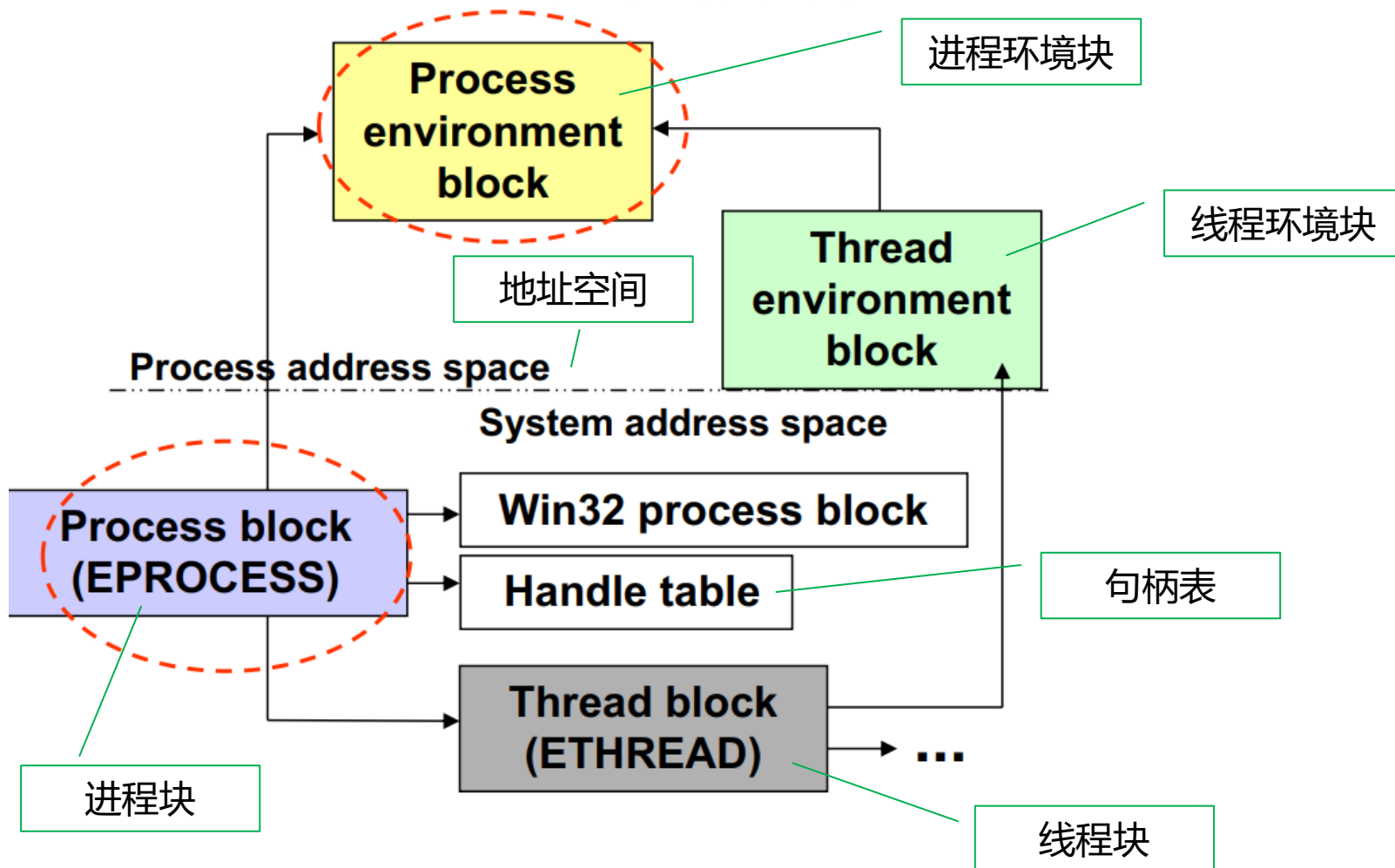
进程 (Process) 的基本概念

★ 进程控制块(Process Control Block, PCB)

信息类别	图片1内容	图片2对应字段
标识信息	进程ID	Process ID (进程ID)
	创建时间	Time when process started (进程启动时间)
	家族树指针	Parent process (父进程)
现场信息	CPU通用寄存器	Program counter (程序计数器) , Stack pointer (栈指针) , Program status word (程序状态字)
控制信息	进程状态	Process state (进程状态)
	优先级	Priority (优先级)
	调度信息队列指针	Scheduling parameters (调度参数)
	资源占用	隐含在CPU time used (已用CPU时间) 中

进程 (Process) 的基本概念

Windows进程数据结构



进程 (Process) 的基本概念

EPROCESS 结构体

```
nt!_EPROCESS
+0x000 Pcb          : _KPROCESS
+0x078 ProcessLock  : _EX_PUSH_LOCK
+0x080 CreateTime   : _LARGE_INTEGER
0x1c9a12b'a6109f10
+0x088 ExitTime     : _LARGE_INTEGER 0x0
+0x090 RundownProtect : _EX_RUNDOWN_REF
+0x094 UniqueProcessId : 0x000000c0
+0x098 ActiveProcessLinks : _LIST_ENTRY
[ 0x808a6f40 - 0x811be338 ]
+0x0a0 QuotaUsage    : [3] 0x320
+0x0ac QuotaPeak     : [3] 0x320
+0x0b8 CommitCharge  : 0x31
+0x0bc PeakVirtualSize : 0x6cd000
+0x0c0 VirtualSize   : 0x6cd000
+0x0c4 SessionProcessLinks : _LIST_ENTRY
[ 0xfa12c010 - 0x811be364 ]
+0x0cc DebugPort     : (null)
+0x0d0 ExceptionPort : 0xe1292bc0
+0x0d4 ObjectTable    : 0xe1138f40
_HANDLE_TABLE
.....
```

// 内核进程控制块
// 进程锁
// 创建时间戳

// 退出时间(未退出时为0)
// 进程终止保护
// 进程ID(PID)
// 活动进程链表

// 资源配额使用量

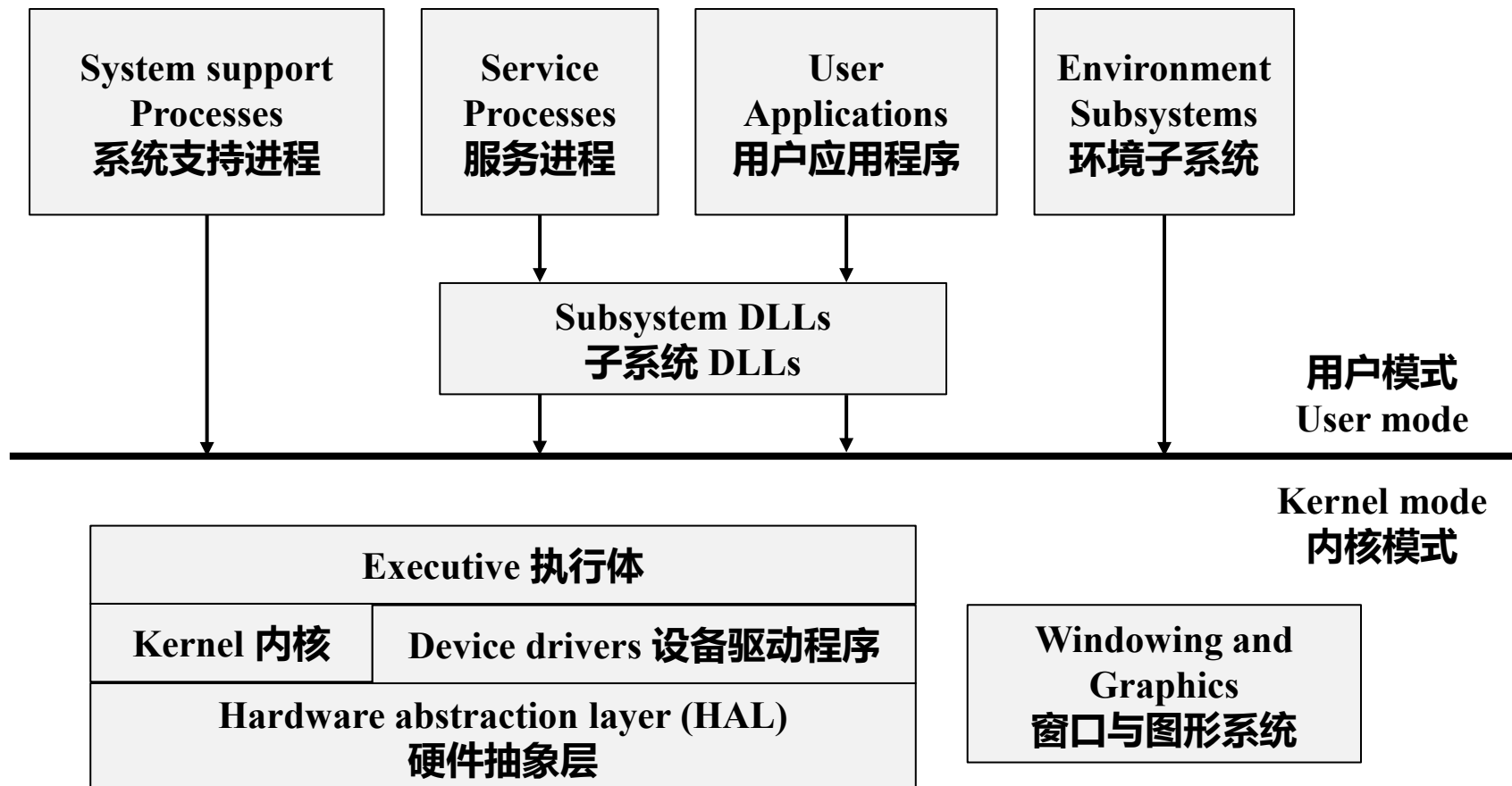
// 提交内存量
// 峰值虚拟内存大小

// 会话进程链表

// 异常处理端口
// 对象句柄表

进程 (Process) 的基本概念

简化的 windows 结构图



进程 (Process) 的执行与控制

进程控制

- ❖ 系统对进程的控制和管理是通过操作系统内核中的原语实现的。
- ❖ 原语
 - 由若干条机器指令构成的可完成特定功能的程序段，它是一个“原子操作(atomic operation)”过程，执行过程不能被中断——要么全都完成，要么全都不做。
 - 原语的原子性主要是通过屏蔽中断保证的。
- ❖ 原语分类
 - 进程控制原语 创建、撤销
 - 进程通信原语 阻塞、唤醒
 - 资源管理原语
 - 其他 挂起、激活

进程 (Process) 的执行与控制

★ 进程相关操作

1. 创建 (Create)

属于进程控制原语。**从分配进程标识、初始化 PCB (进程控制块)、申请资源等步骤**，需一气呵成（原子性）。比如系统为新进程分配内存时，不能中途被中断，否则内存分配混乱，后续进程管理也会出错，得“要么全完成（创建好进程），要么全不做（回退资源分配，不生成无效进程）”

2. 撤销 (Destroy)

同样是进程控制原语。**要回收进程占用的所有资源（如内存、打开的文件等）、更新系统进程表等**。若执行中被中断，残留资源会导致泄漏，影响系统稳定性，所以得靠原语保证“彻底回收（全完成）”或“不开始回收（全不做）”，避免资源混乱

3. 阻塞 (Block)

是进程控制原语。**当进程等待I/O、信号量等资源/事件时，需修改进程状态为阻塞态、将其从就绪队列移除等操作**。这些步骤必须连续，若中途被中断，进程状态和队列管理就会出错，原语保证“状态变更+队列调整”完整执行，让进程正确进入阻塞等待

进程 (Process) 的执行与控制

★ 进程相关操作

4. 唤醒 (Wakeup)

属于进程控制原语。**针对阻塞态进程，满足条件后要修改其状态为就绪态、重新加入就绪队列**。这一系列操作得“原子”执行，否则进程状态可能异常（既非阻塞也非就绪），队列也乱套，原语确保“状态改+入队”一步到位，让进程能被调度运行

5. 挂起 (Suspend)

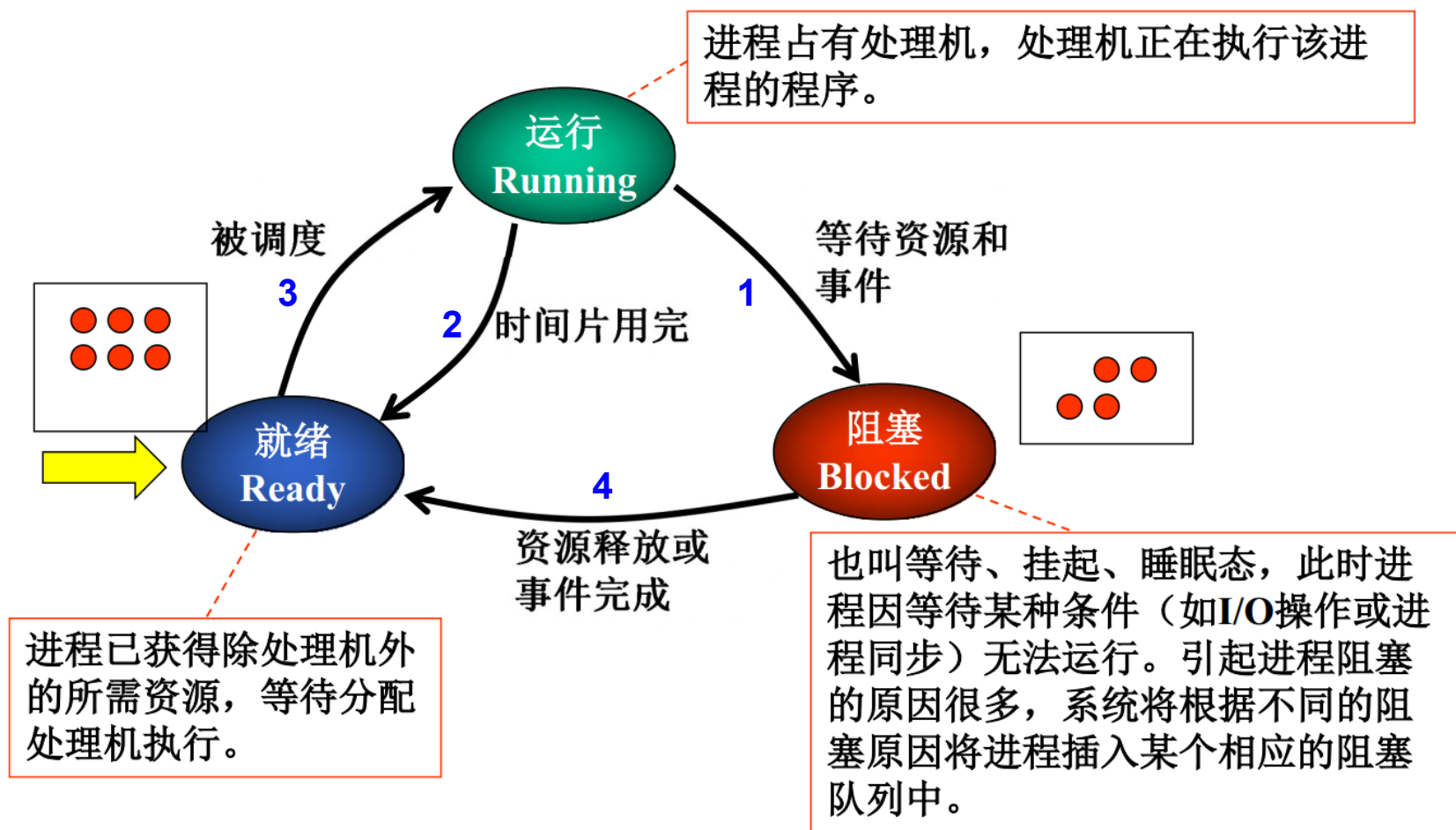
作为进程控制原语，**常用于将进程暂时移出内存（如交换到外存）、标记挂起状态**。操作涉及状态修改、内存资源转移等，必须连续。若中断，进程可能“半在内存半在外存”，状态标识也混乱，原语保证要么完整挂起（资源转移+状态更新），要么不执行，维持系统资源管理有序

6. 激活 (Activate)

属于进程控制原语，**是挂起的逆操作。要把挂起进程从外存调回内存、恢复就绪/运行相关状态**。执行时需“原子性”，保证进程资源完整恢复、状态正确更新，否则进程可能因资源不全、状态错误无法正常执行，原语让“调回内存+状态激活”连贯完成，使进程回归可调度序列

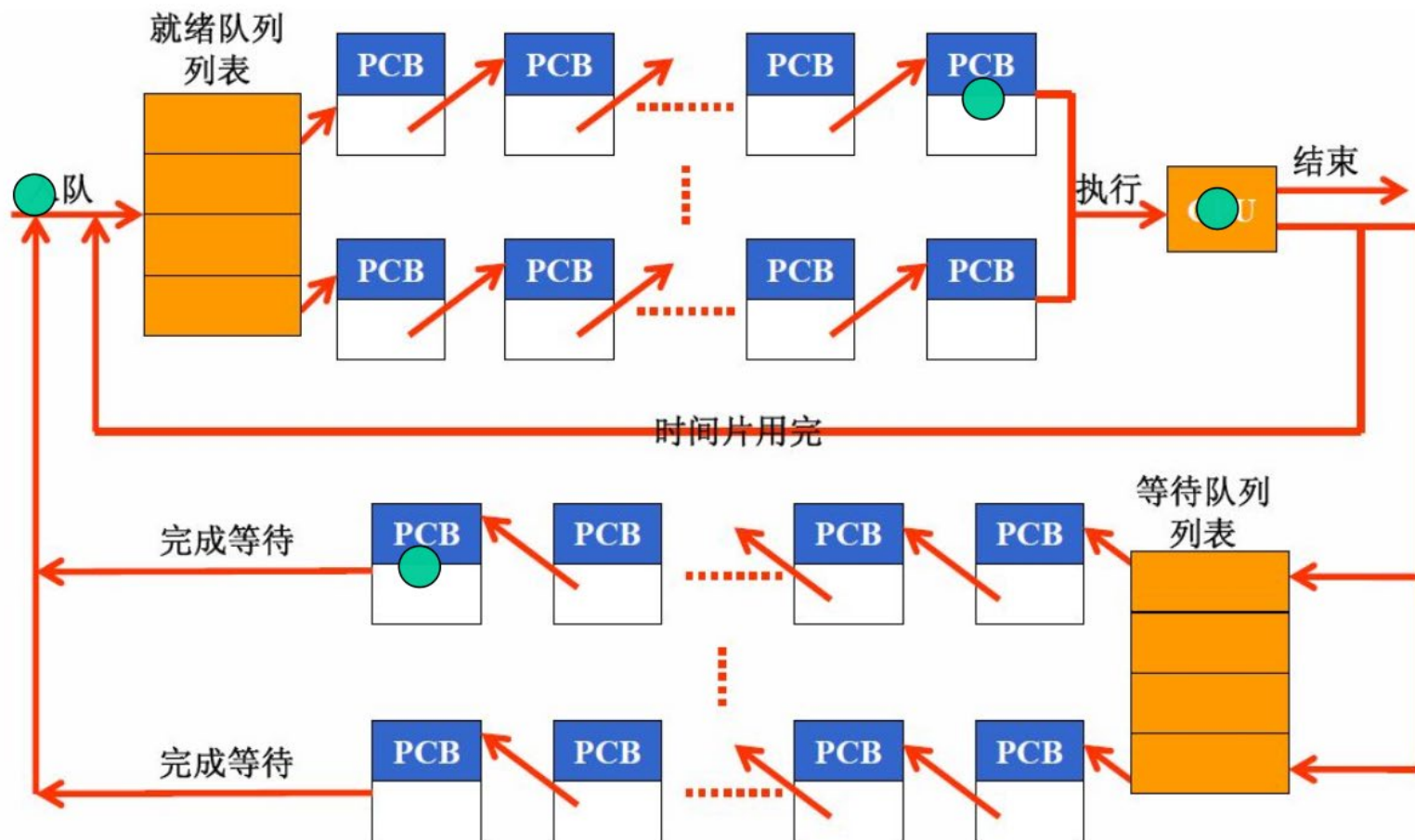
进程 (Process) 的执行与控制

进程的基本状态及其转换



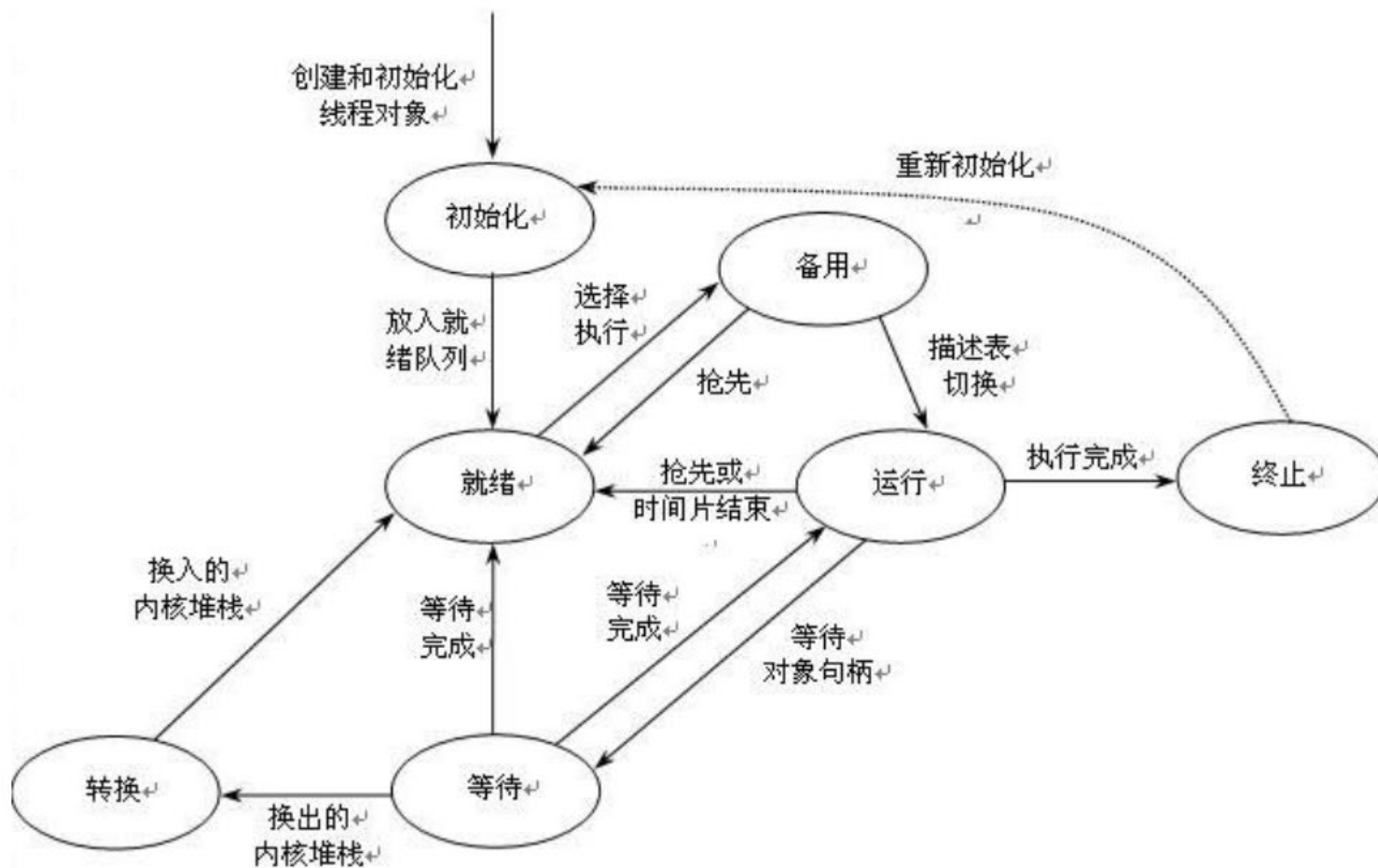
进程 (Process) 的执行与控制

进程的组织管理——队列



进程 (Process) 的执行与控制

Windows 2000/XP 进程状态



进程与线程——考纲内容

(一) 进程与线程

- 进程与线程的基本概念；进程 / 线程的状态与转换
- 线程的实现：内核支持的线程，线程库支持的线程进程与线程的组织与控制
- 进程间通信：共享内存，消息传递，管道，信号

(二) CPU 调度与上下文切换

- 调度的基本概念；调度的目标；
- 调度的实现：调度器 / 调度程序 (scheduler)，调度的时机与调度方式（抢占式 / 非抢占式），闲逛进程，内核级线程与用户级线程调度
- CPU 调度算法
- 多处理机调度
- 上下文及其切换机制

进程与线程——考纲内容

(三) 同步与互斥

- 同步与互斥的基本概念
- 基本的实现方法：软件方法；硬件方法
- 锁；信号量；条件变量
- 经典同步问题：生产者 - 消费者问题，读者 - 写者问题；哲学家进餐问题

(四) 死锁

- 死锁的基本概念；死锁预防
- 死锁避免；死锁检测和解除

【复习提示】进程管理是操作系统的核心，也是每年必考的重点。其中，进程的概念、进程调度、信号量机制实现同步和互斥、进程死锁等更是重中之重，必须深入掌握。需要注意的是，除选择题外，本章还容易出综合题，其中信号量机制实现同步和互斥、进程调度算法和死锁等都可能命制综合题，如利用信号量进行进程同步就在往年的统考中频繁出现。

回顾：进程（Process）的基本概念

进程(process)的定义

❖ 进程的提出：

- IBM: CTSS/360系统，称为“工作/任务” (Job/Task)。
- 60年代初，MIT: MULTICS系统首先引入“进程”的概念；

❖ 尚无公认的统一定义。

❖ 直观的定义

- 进程就是进展中的程序 / 执行中的程序

进程 = 程序 + 执行

❖ 教材中：

- 进程是程序的一次执行，该进程可与其它进程并发执行；它是一个动态的实体，在传统的操作系统设计中，进程既是资源的基本分配单元，(也是基本的执行单元)。

回顾：进程（Process）的基本概念

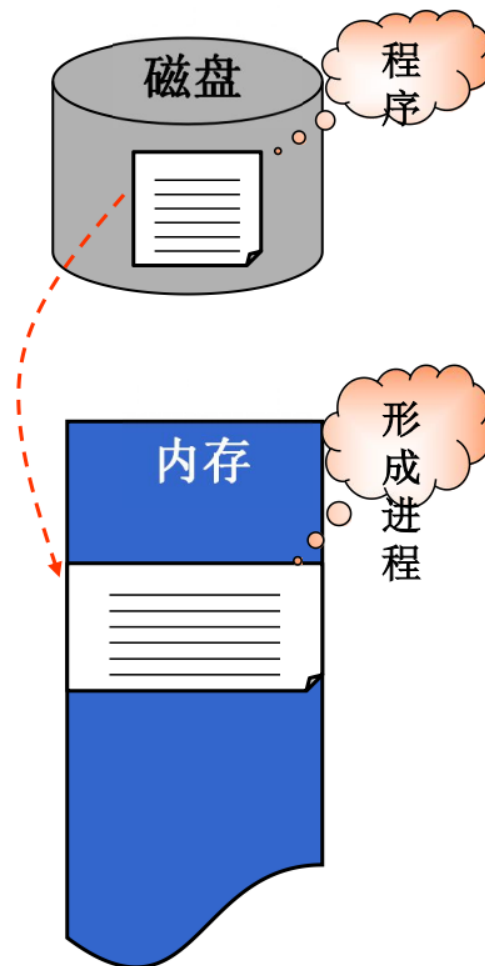
进程与程序的区别和联系

❖ 区别：

- 程序是**静态**的，是有序代码的集合；进程是**动态**的，是程序的一次执行。
- 程序是**永久**的，没有生命周期，可长久保存；进程是**暂时**的，有生命周期，是一个动态不断变化的过程。
- 进程是操作系统**资源分配和保护的基本单位**，程序没有此功能。
- 进程与程序的结构不同。

❖ 联系：

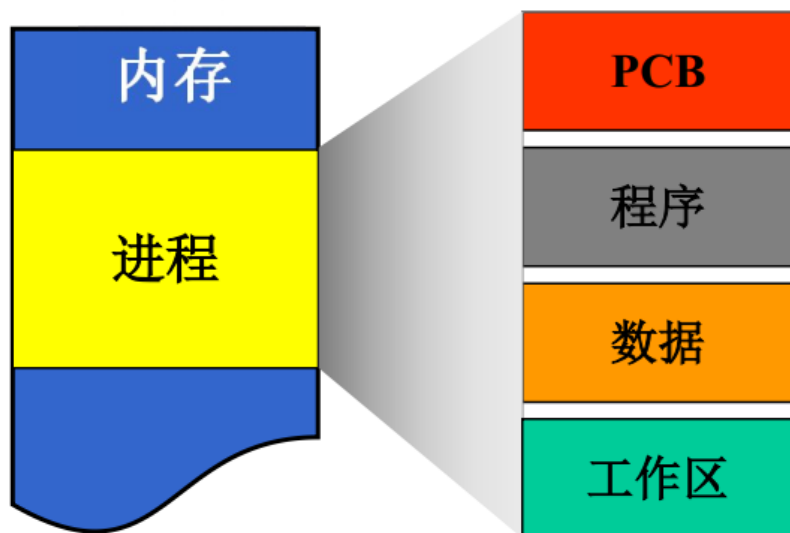
- 通过多次执行，一个程序可对应多个进程；通过调用关系，一个进程可包括多个程序。



回顾：进程（Process）的基本概念

进程的组成

❖ 进程的存储映像



“Process Control Block”，包含着进程的描述和控制信息，进程存在的唯一标志。 } 灵魂

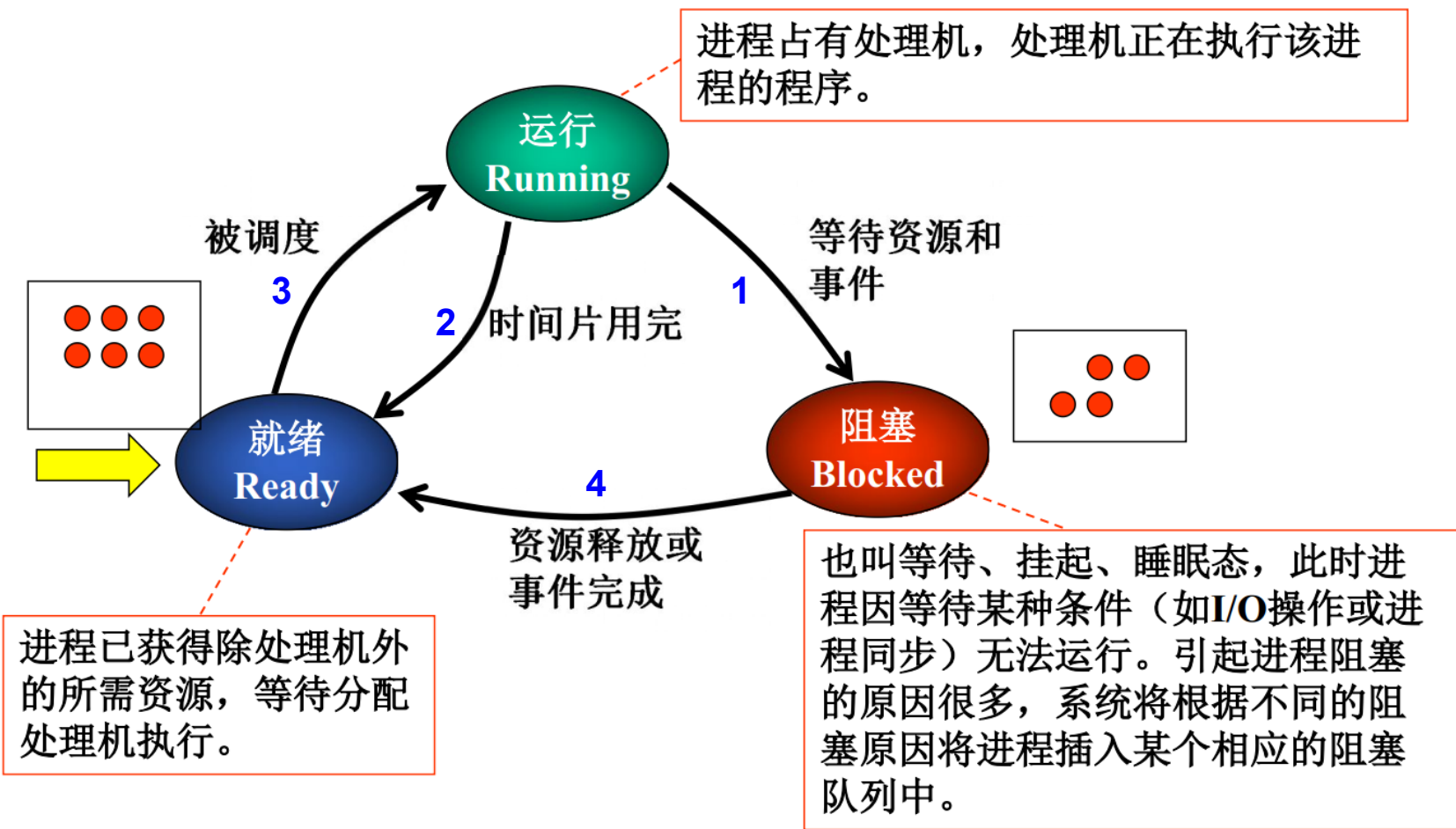
“纯代码”部分，描述了进程要完成的功能，是进程执行时不可修改的部分。 } 实体

进程执行时用到的数据。

一片可供进程使用的动态区域(堆栈区)，
可用于：保存局部变量，传递参数、系统调用时存放返回地址等。

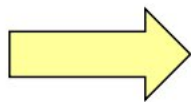
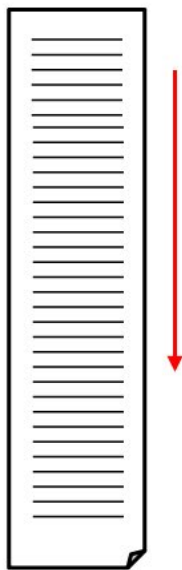
回顾：进程（Process）的执行与控制

进程的基本状态及其转换



线程 (Thread)

为何要有线程？



进程——孤独的舞者
一次只能做一件事

线程——进程的分身术

线程 (Thread)

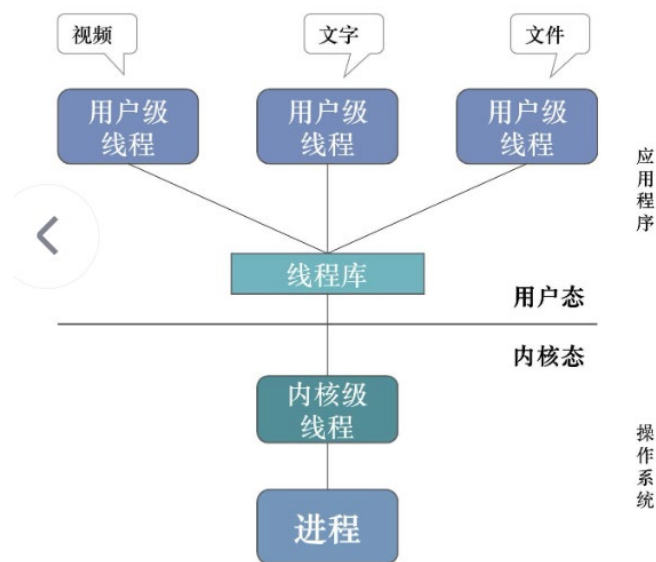
★ 多线程进程(餐厅): 多个厨师(线程)共享资源, 并行处理任务



线程 (Thread)

线程的定义

- ❖ 线程(Thread)也叫轻型进程，是一个可执行的实体单元，是现代操作系统中处理机调度(执行)的基本单位。
- ❖ 线程和进程的关系

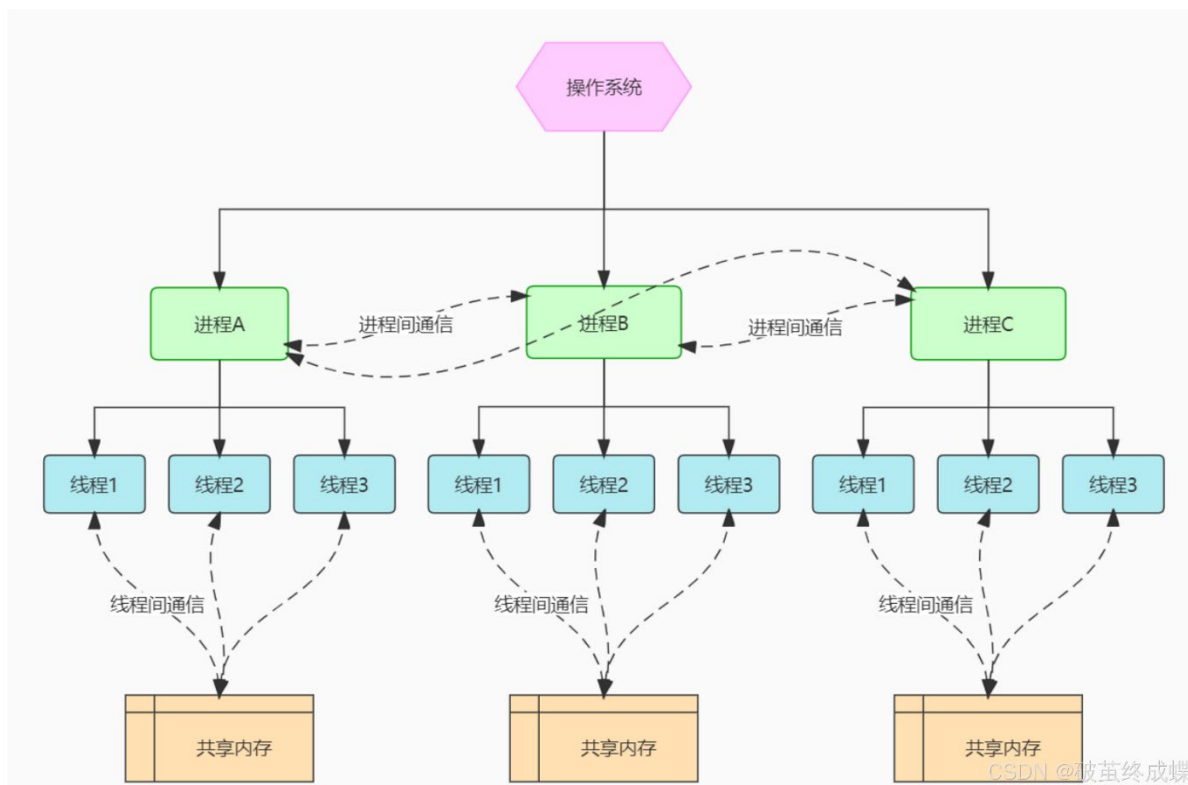


① **线程是进程的一个组成部分**，线程由进程创建，因此一个进程中至少存在一个线程，线程还可以创建其它线程；

② **进程是资源分配和保护的基本单位**，线程只能在进程的地址空间活动，线程只能使用其所在进程的资源。

线程 (Thread)

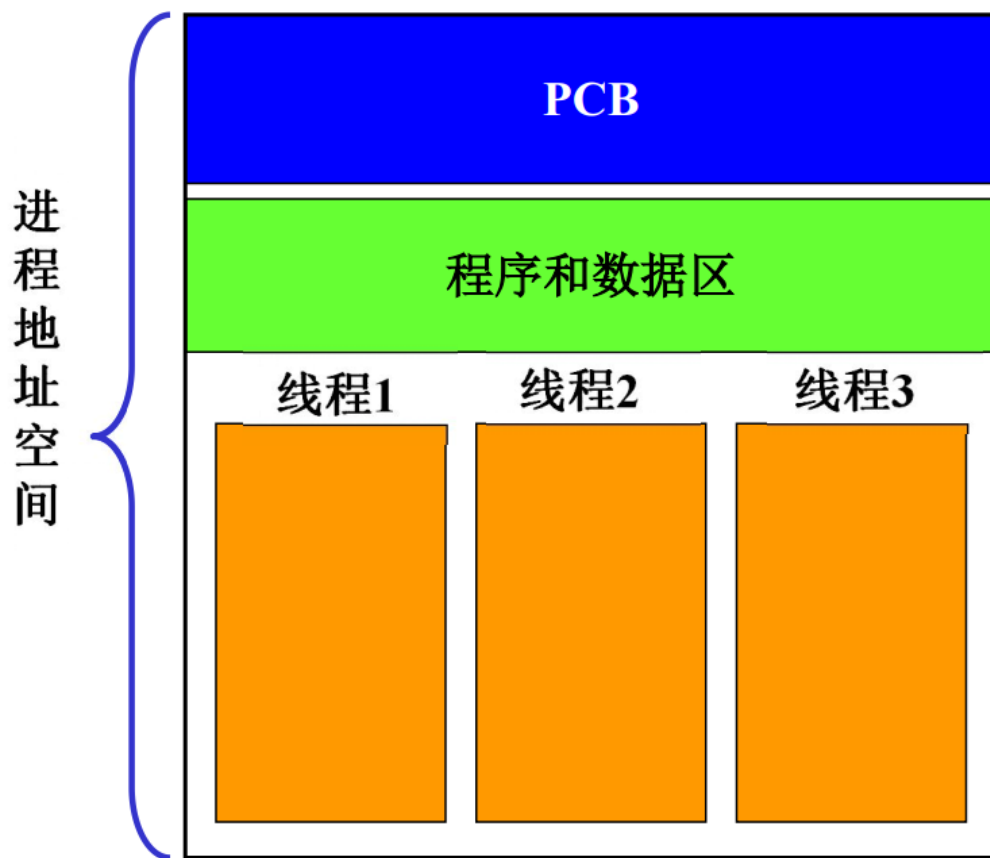
❖ 线程和进程的关系



- ①所有线程拥有**相同的程序文本**，不同的线程可执行相同或不同的代码段；
- ②线程**可独立运行，也可相互合作**；
- ③每个线程拥有**独立上下文 (运行环境)**

线程 (Thread)

线程的结构和特征



线程共享的资源:

- 地址空间
- 全局变量
- 打开的文件
- 子进程
- 信号量
- ...

线程独享的信息:

- 程序计数器
- 寄存器
- 栈
- 状态字
- TCB

线程 (Thread)

★ 线程共享资源

是线程能协作完成任务的基础，本质是 “**进程级公共资源**，线程无需重复申请”

- **地址空间**：**进程的内存范围**（代码段、数据段、堆等），线程执行时直接访问，避免重复分配内存，让线程间数据交互更高效（如全局变量共享）
- **全局变量**：**进程级的变量**，所有线程可读写（需同步机制保护），是线程协作传数据的“公共黑板”
- **打开的文件**：进程**打开的文件句柄**，线程可直接操作（读/写），比如多线程下载同一个文件，共享文件描述符避免重复打开
- **子进程**：**进程创建的子进程**，线程可共同管理（如多线程监控子进程状态），子进程资源也归属父进程地址空间，自然被线程共享
- **信号量**：**进程级的同步工具**，线程用它协调对共享资源的访问（比如控制线程A读完数据，线程B再处理），是线程“有序协作”的开关

线程 (Thread)

★ 线程独享信息

是线程独立运行的“私有执行上下文”，保证线程切换时不出错、执行逻辑不冲突

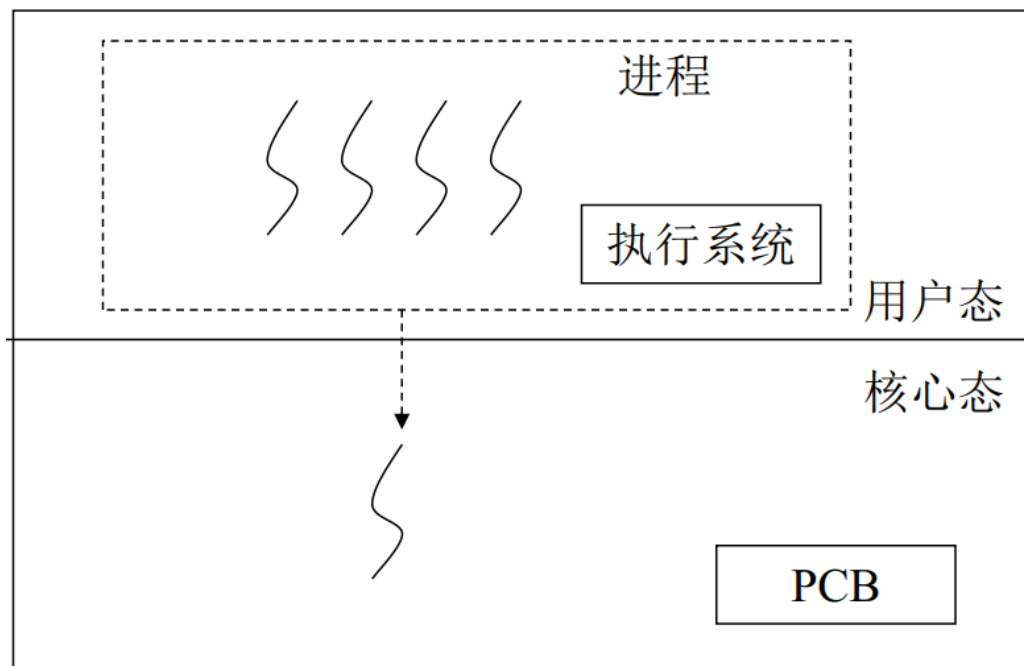
- **程序计数器 (PC)**：记录线程**下一条要执行的指令地址**。线程切换时，需保存/恢复PC，否则CPU不知道“接下来该执行哪行代码”
- **寄存器**：**CPU临时存数据的高速单元**（如eax、ebx寄存器），线程执行时会用到（如计算临时变量）。切换线程必须保存/恢复寄存器值，否则数据会混乱（如线程A的计算结果被线程B覆盖）
- **栈**：线程的“**私有调用栈**”，存函数参数、局部变量、返回地址。多线程同时执行不同函数时，栈独立能避免数据干扰（比如线程1的函数调用栈不会影响线程2）
- **状态字**：**记录线程执行状态**（运行、就绪、阻塞），操作系统靠它调度线程（比如优先调度“就绪态”线程）
- **TCB (线程控制块)**：**线程的“身份档案”**，存线程ID、优先级、栈指针等信息，是操作系统管理线程的核心结构（类似进程的PCB，但粒度更细）

线程 (Thread)

线程的实现方式

- ❖ 线程管理者的选择：操作系统？进程？
- ❖ 两种不同的线程实现方式：内核态线程；用户态线程。

用户态线程



优点：

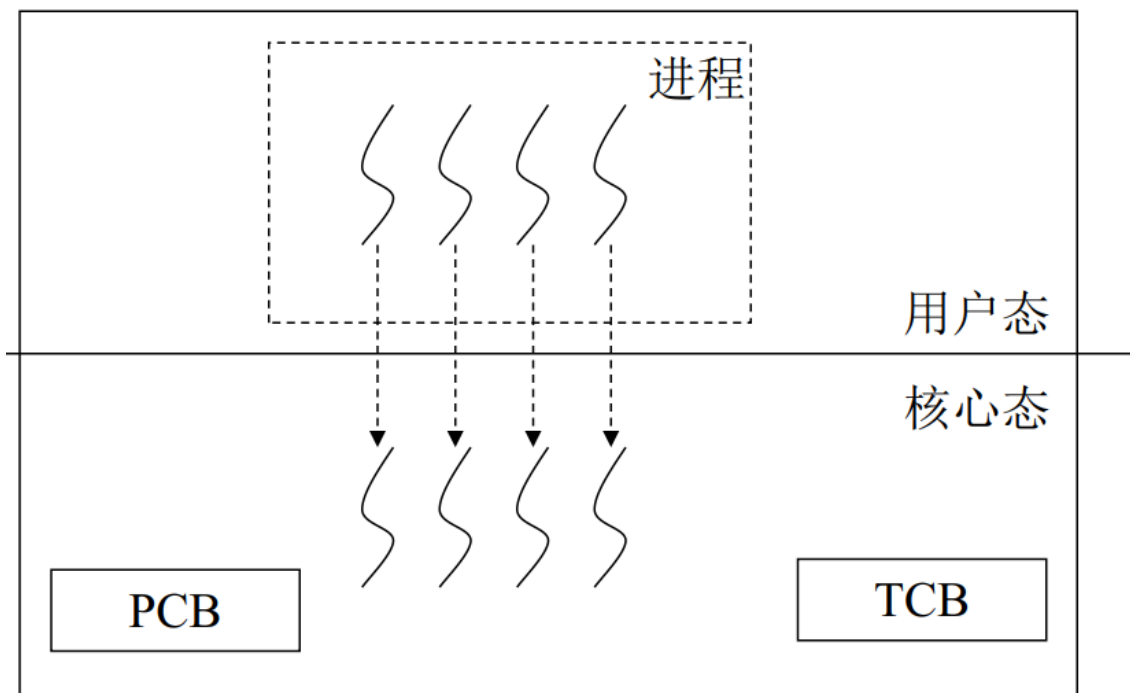
- 操作系统不再能感知到用户线程的存在，操作系统适应性强。
- 线程间切换很迅速

缺点：

- 程序设计难度加大
- 并发度降低

线程 (Thread)

内核态线程

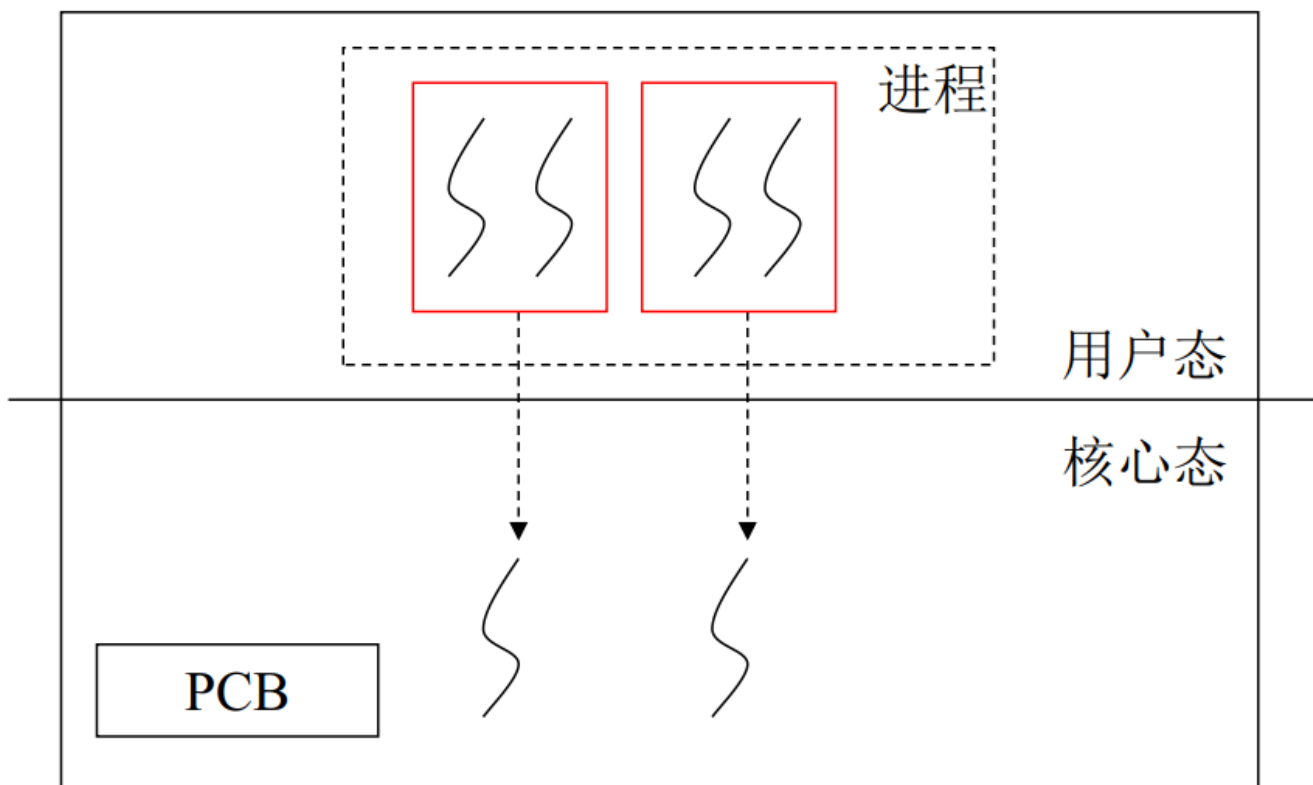


优点：实现简单，并发度高

缺点：执行效率低，
消耗资源量大

线程 (Thread)

混合实现模型——一种折中的选择



进程 (Process) 的调度 (Schedule)

- Problem

- When multiple processes/threads are ready, which one should be put on the CPU?
 - Sub Problem1: When to make such decision?
 - Sub Problem 2: What is the decision?

- Scheduling in Batch Systems
- Scheduling in Interactive Systems
- Scheduling in RT Systems
- Other
 - Policy Versus Mechanism
 - Thread Scheduling

- 问题

- 当多个进程 / 线程就绪时, 应将哪一个放到 CPU 上执行?
 - 子问题 1: 何时做出这样的决策?
 - 子问题 2: 依据什么来做决策?

- 批处理系统中的调度
- 交互式系统中的调度
- 实时系统中的调度
- 其他
 - 策略与机制
 - 线程调度

进程 (Process) 的调度 (Schedule)

When to Schedule

- Process Creates
- Process exits
- Process blocks on I/O
- I/O interrupt

What is the decision

- Scheduling Algorithm
- Category??--- The benevolent see benevolence and the wise see wisdom
 - Preemptive vs. Non-Preemptive
 - Batch
 - Interactive
 - Real time

何时调度

- 进程创建
- 进程退出
- 进程因 I/O 阻塞
- I/O 中断

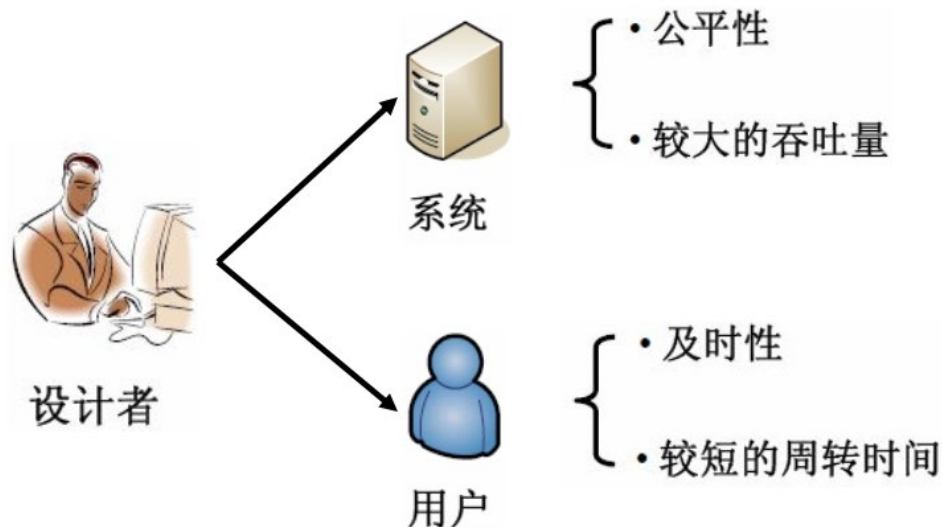
依据什么决策

- 调度算法
- 分类?? —— 仁者见仁, 智者见智
 - 抢占式与非抢占式
 - 批处理
 - 交互式
 - 实时

进程 (Process) 的调度 (Schedule)

- ★ **定义：** 依托特定算法，从就绪队列里挑选进程，让其获得 CPU 使用权
- 决定 CPU 资源在就绪进程间如何分配，是 OS 管理 CPU 资源的关键环节

- ★ **原则：**



时机、机制、过程
When、How、What

进程 (Process) 的调度 (Schedule)

★ 原则:

1. 调度时机 (When)

指什么情况下会触发进程调度，也就是CPU使用权该重新分配的“触发条件”，常见场景：

➤ 进程状态变化时:

- 运行→阻塞（比如进程发起I/O请求，要等硬盘读数据），CPU空出来，得选新进程运行
- 运行→就绪（比如进程用完时间片，或被更高优先级进程抢占），也会触发重新调度

➤ 系统事件发生时:

- 进程创建完成（新进程加入就绪队列，可能插队调度）
- 系统调用结束、中断处理完成（比如时钟中断定期触发，检查是否该换进程）

简单说：就是**CPU空闲了**、或**系统需要重新分配资源时**，就会触发调度“时机”，决定下一步哪个进程运行

进程 (Process) 的调度 (Schedule)

★ 原则:

2. 调度机制 (How)

指用什么“规则/算法”选进程，是调度的“核心逻辑”，解决“选谁”的问题。

➤ 常见算法 (机制) :

- 先来先服务 (FCFS) : 按进程进入就绪队列的顺序, 排队选, 像银行叫号
- 短作业优先 (SJF) : 优先选预估运行时间最短的进程, 提升整体吞吐量
- 时间片轮转 (RR) : 给每个进程分配固定时间片, 用完就切换, 保障公平性
- 优先级调度: 按进程优先级 (系统或用户设定), 高优先级先运行

进程 (Process) 的调度 (Schedule)

★ 原则:

3. 调度过程 (What)

指调度算法执行时，具体做了哪些操作，是“选进程+切换上下文”的完整流程

➤ 一般分两步:

- 选择进程：用上面说的“调度机制（算法）”，从就绪队列挑出下一个运行的进程
- 上下文切换：保存当前运行进程的“上下文”（比如CPU寄存器数据、程序计数器、内存状态等），存在PCB（进程控制块）里；加载选中进程的“上下文”，恢复它上次暂停时的状态，让CPU接着执行

总结： **时机**是“触发调度的条件”（**什么时候**换进程）； **机制**是“选进程的规则”（用**什么算法**选）； **过程**是“选进程+切换状态的操作”（具体**怎么换**）

进程 (Process) 的调度 (Schedule)

先来先服务调度算法 (FCFS, First Come First Served)

❖ 思想:

- 先来先服务。

√公平性

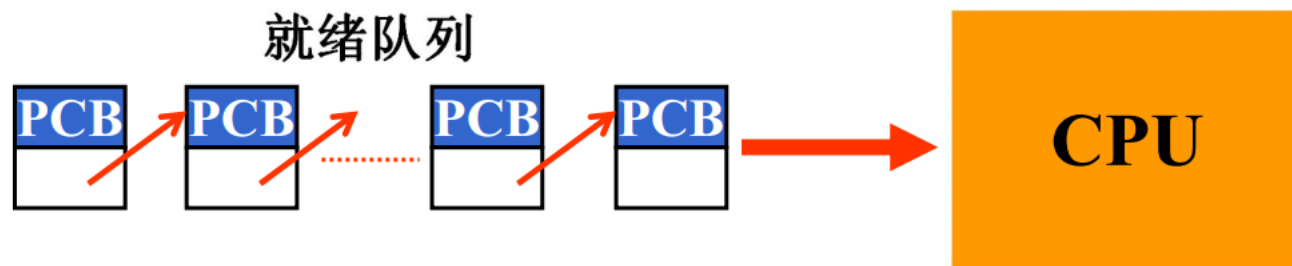
❖ 特点:

- 简单
- 可靠
- 实现方便
- 灵活性差。

×吞吐量

×及时性

×周转时间

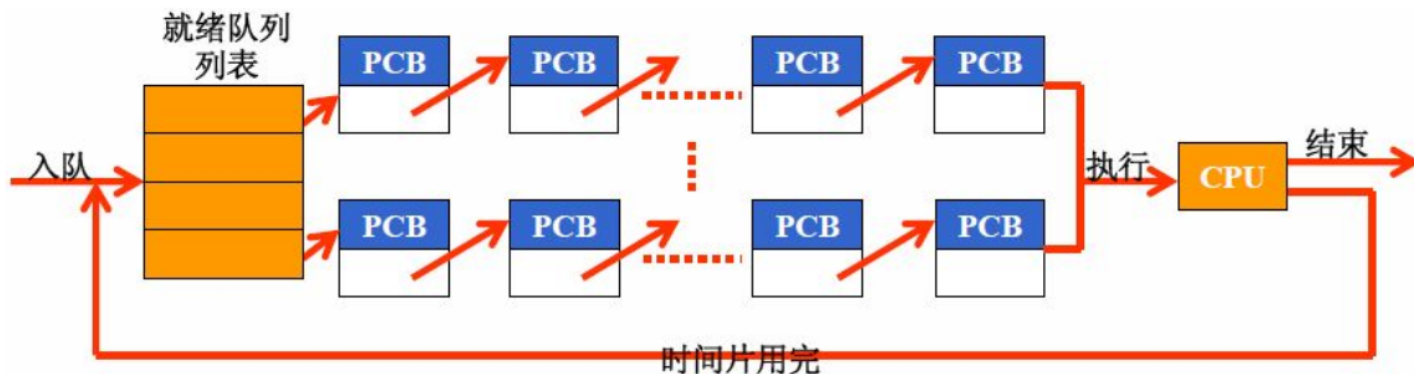


也称为 先进先出 (FIFO, First In First Out)

进程 (Process) 的调度 (Schedule)

时间片轮转法(RR, Round Robin)

- ❖ 思想：系统规定一个时间长度(时间片/时间量)作为允许一个进程运行的时间，如果在这段时间该进程没有执行完，则必须让出CPU等待下一次分配的时间片。



× 公平性

× 吞吐量

√ 及时性

× 周转时间

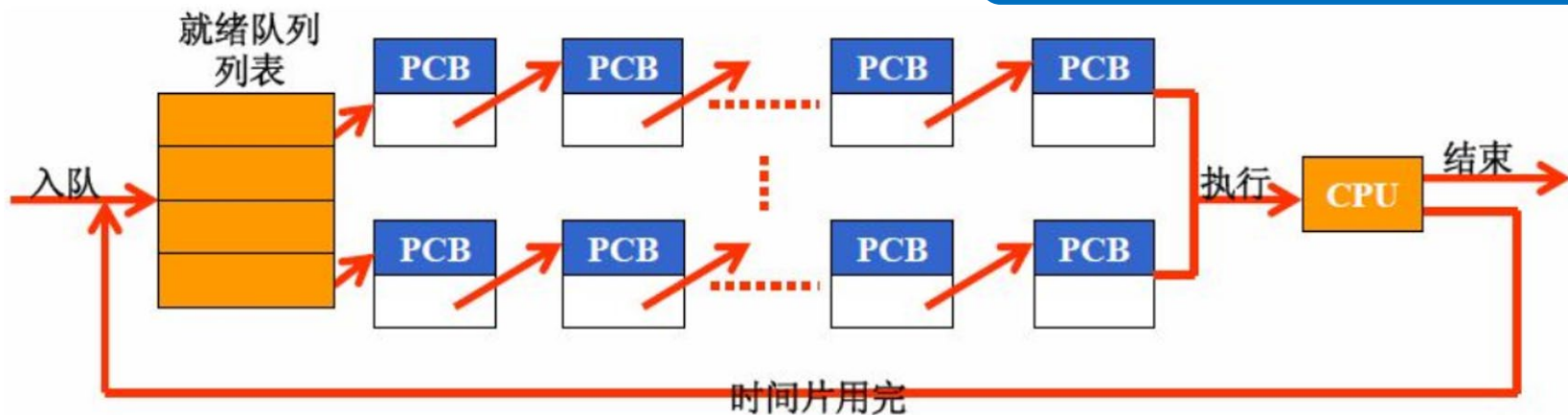
- ❖ 特点：

- 专门为分时系统设计，较为公平性，对用户反应及时。

进程 (Process) 的调度 (Schedule)

时间片轮转法(RR, Round Robin)

为什么叫 **Round Robin**?



“Round Robin” 这个名字起源于中世纪的法国（法语叫 *ruban rond*，意思是“圆形的带子”），它指的是一种请愿书上的签名方式

进程 (Process) 的调度 (Schedule)

基于优先数的调度算法(Priority Scheduling Algorithm)

❖ 思想:

- 给每一个进程设置一个**优先数**(优先级), 系统在调度时优先选择具有高优先级的进程占用**CPU**。具有相同优先数的进程按照**FCFS**算法执行。

❖ 优先数的确定:

- **静态优先数法**: 进程创建时就规定好它的优先数, 这个数值在进程运行时不变。
- **动态优先数法**: 进程的优先数在执行过程中可以根据情况变化而改变。

Preemptive vs. Non-Preemptive

抢占式 VS. 非抢占式

- "Preemptive" Concepts
- "Non-Preemptive" Concepts

进程 (Process) 的调度 (Schedule)

★ 基于优先数调度与抢占 / 非抢占的关系

基于优先数决定进程调度 “该选谁”，抢占 / 非抢占决定 “何时让权”，二者组合适配不同系统需求。

1. 抢占式 + 优先数

新 / 高优先级进程就绪时，直接抢占 CPU（如实时系统处理紧急任务），保障高优先级任务即时响应。

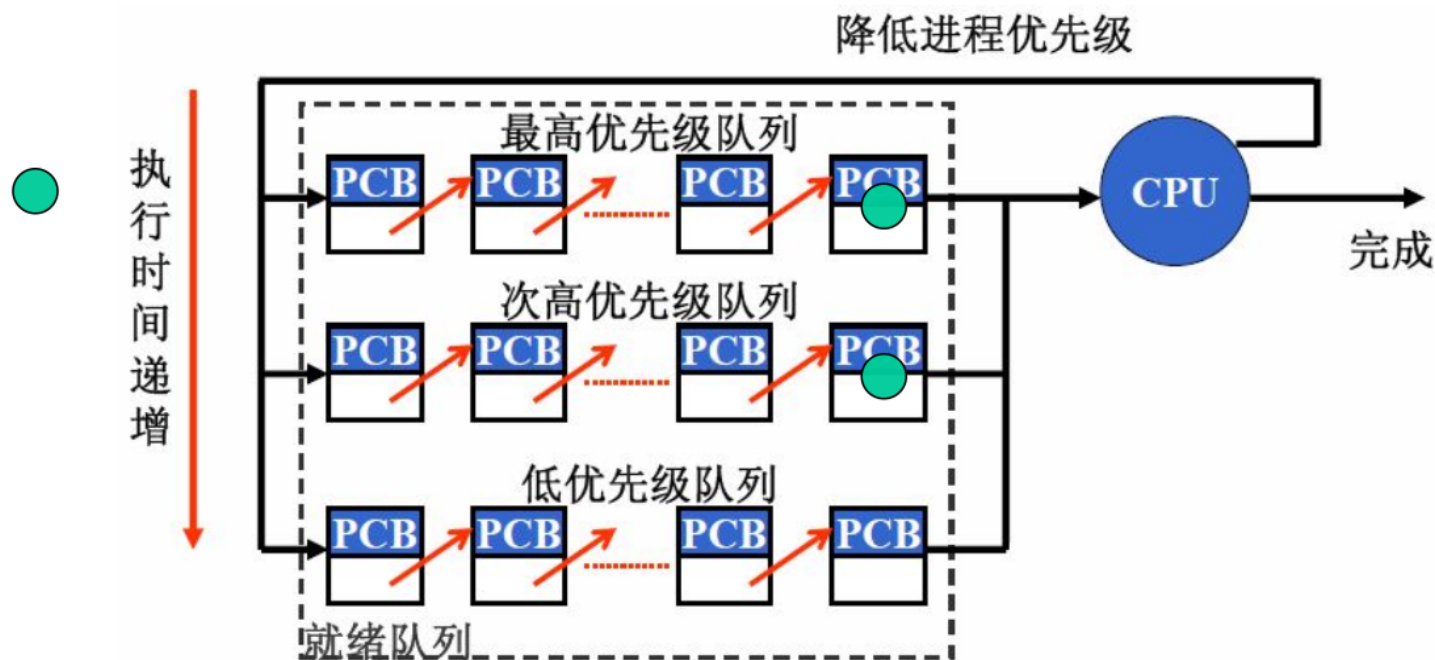
2. 非抢占式 + 优先数

进程一旦获 CPU，运行至结束 / 主动放弃（如批处理系统），简化调度、降低开销，适合对实时性要求低的场景。

进程 (Process) 的调度 (Schedule)

多级反馈队列调度算法(Multilevel Feedback Queue Scheduling)

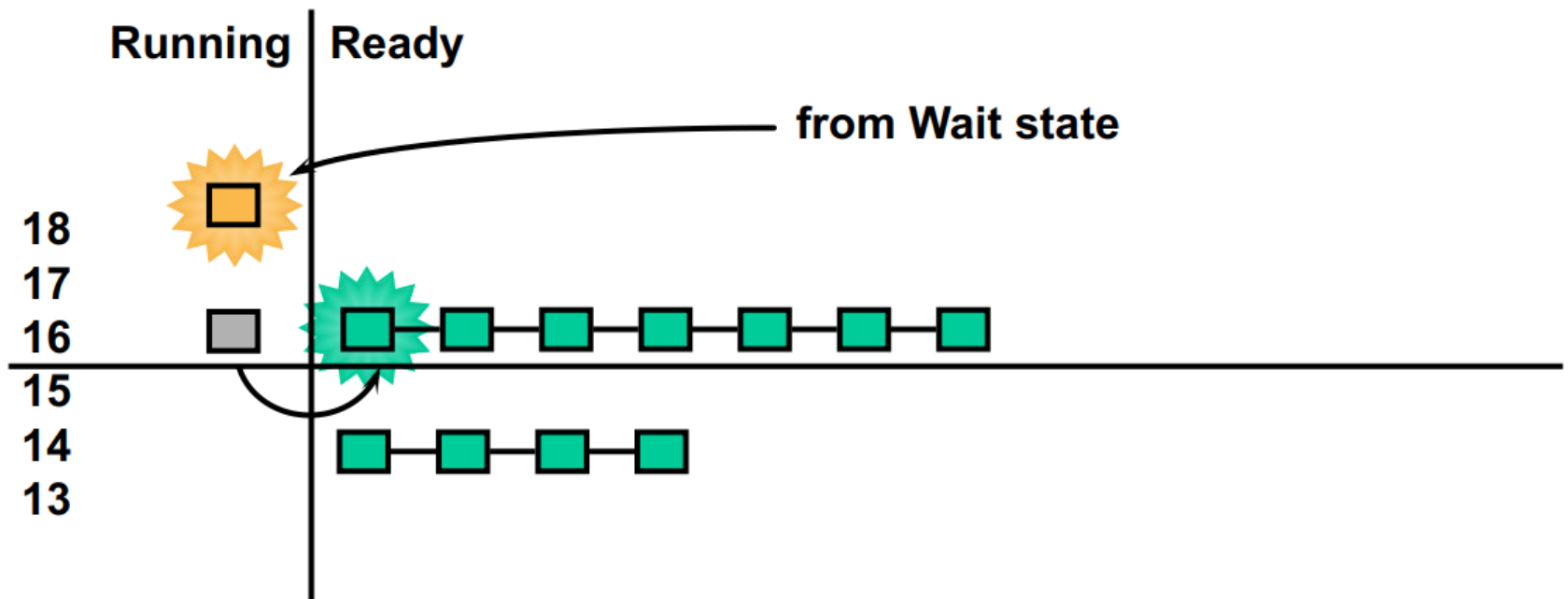
- ❖ 思想：引入多个就绪队列，通过对各队列的区别对待，达到一个综合的调度目标。
- ❖ 原理：



进程 (Process) 的调度 (Schedule)

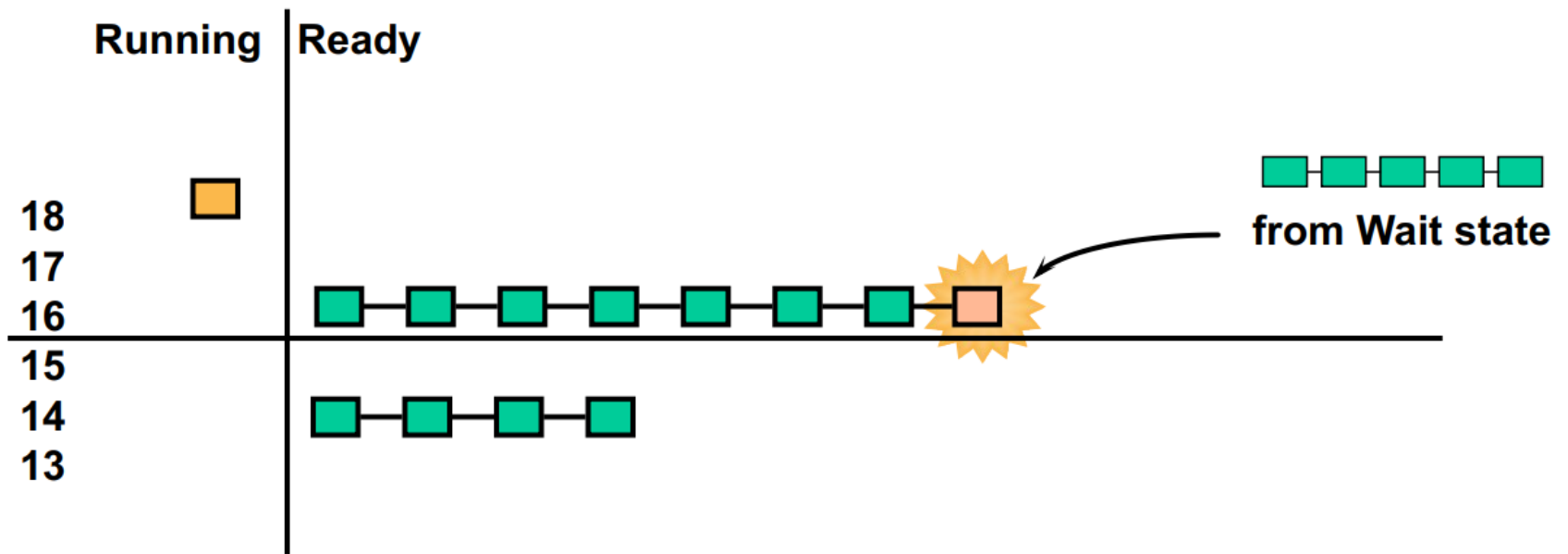
调度情形

❖ Preemption(抢占)



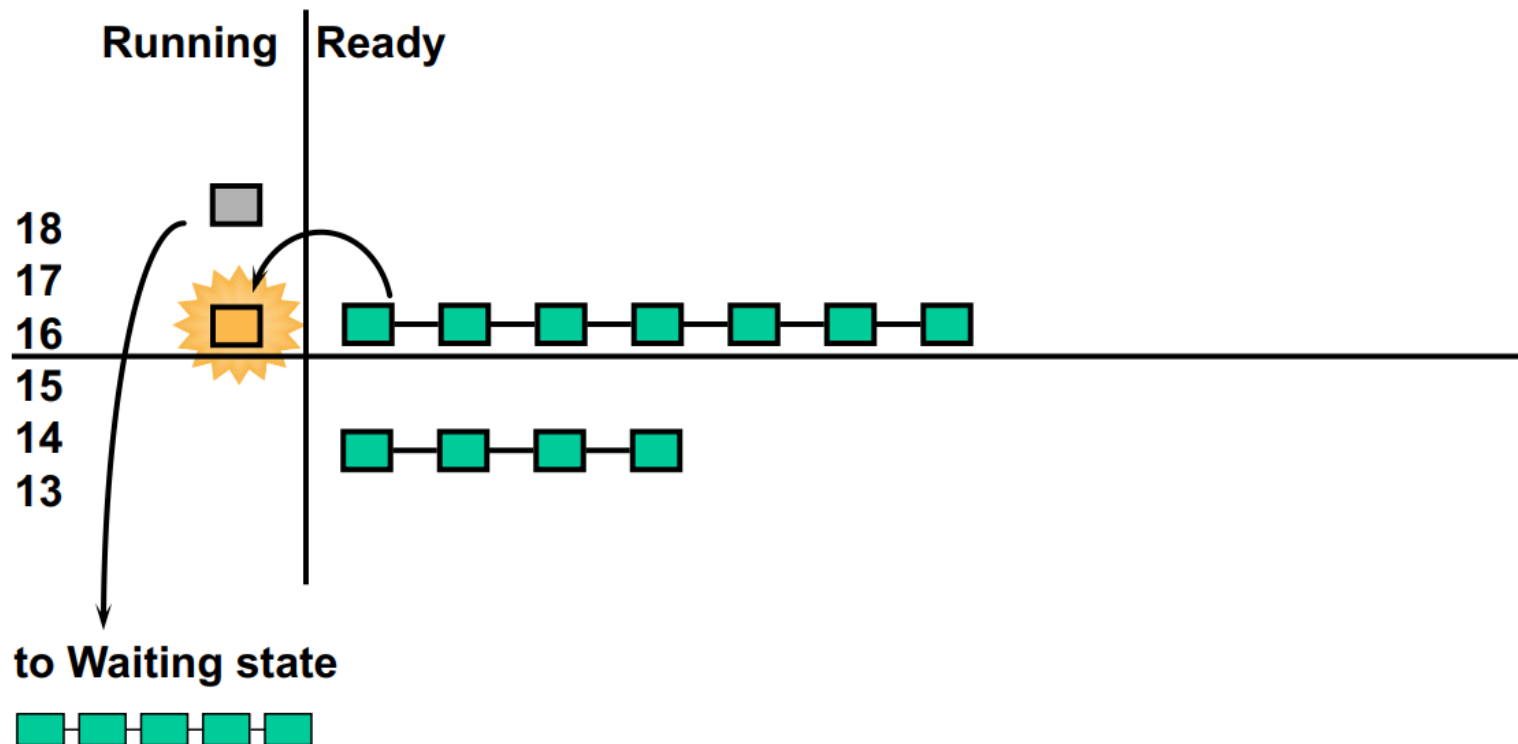
进程 (Process) 的调度 (Schedule)

❖ Ready after Wait Resolution(等待结束进入就绪状态)



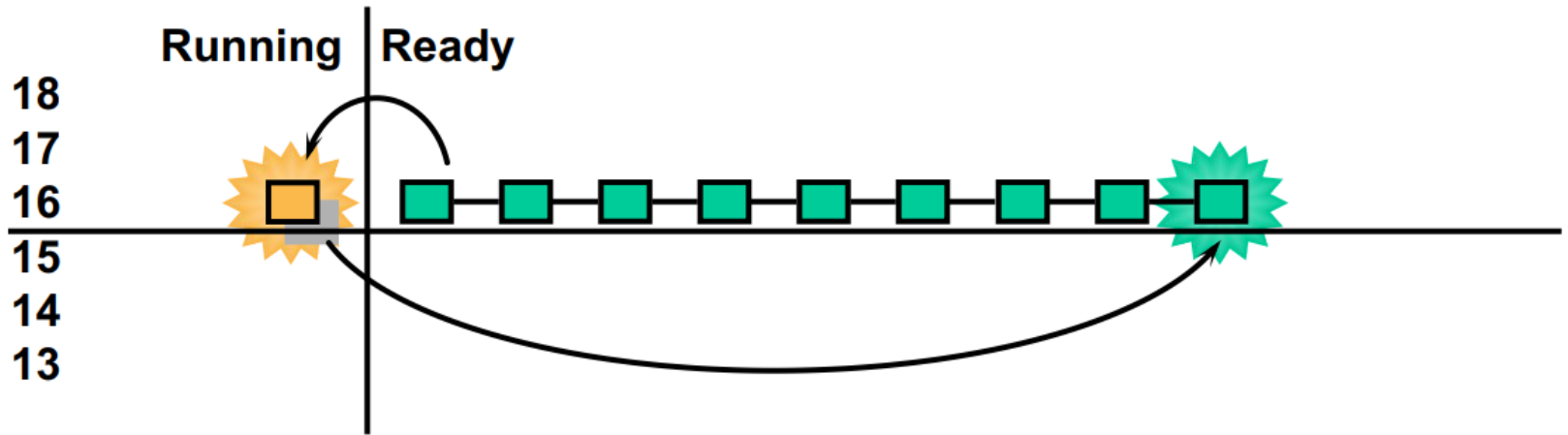
进程 (Process) 的调度 (Schedule)

❖ Voluntary Switch(自愿切换)



进程 (Process) 的调度 (Schedule)

❖ Quantum End (时间片用完)

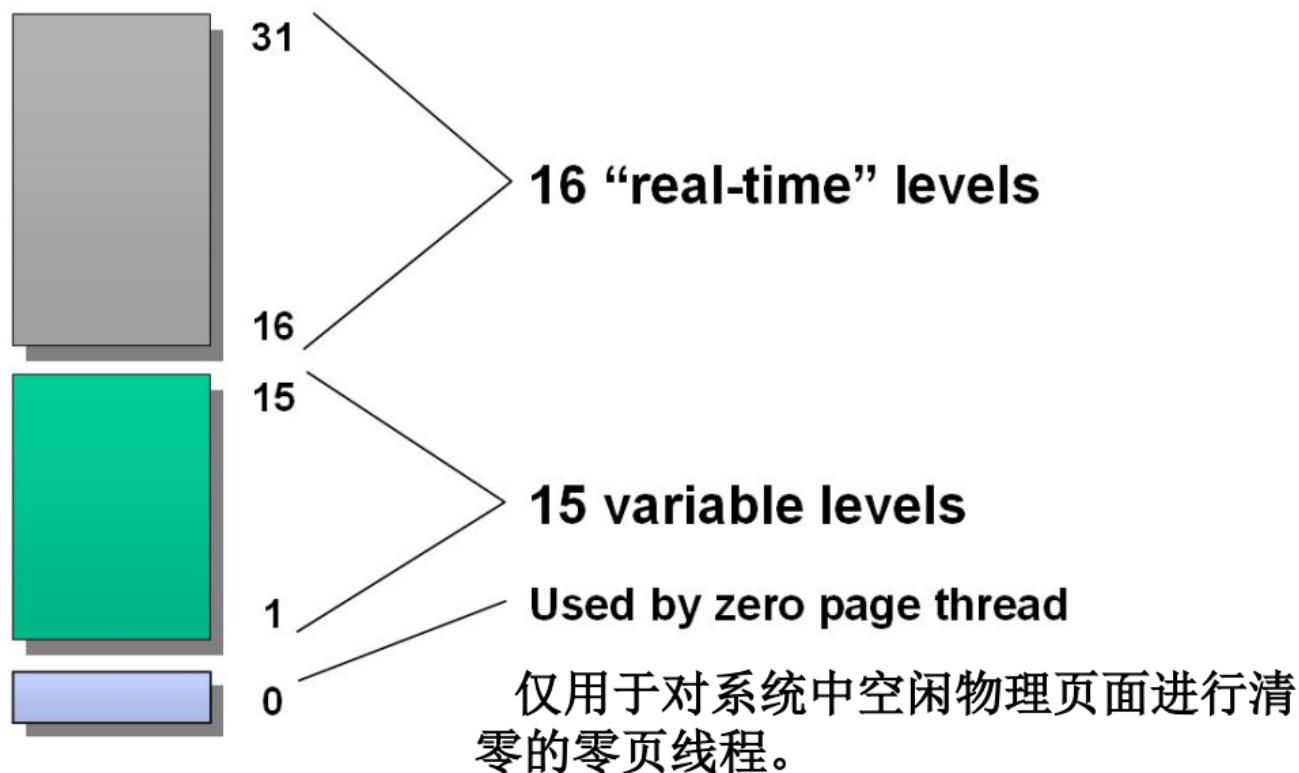


进程 (Process) 的调度 (Schedule)

调度算法(Windows XP/2000/2003)

❖ 采用“动态优先级多队列调度算法”

优先级



进程调度→线程调度

★ 进程

- 进程是资源所有权的载体（拥有内存、文件等）
- 传统调度中，进程作为整体被切换，开销大（切换整个地址空间）
- 进程内部是单一路径执行，无法充分利用多核CPU

★ 为什么需要线程？

- **线程**：“轻量级进程”，它是**CPU调度和执行的基本单位**
- 同一个进程内的多个线程：
共享相同的资源（内存、文件）-> 切换开销极小
独立地运行、被调度（拥有独立的PC、寄存器、栈）-> 并发性更好

★ 调度器的转变：

- **现代操作系统调度的是线程，而不是进程**
- 内核看到的不再是一个个沉重的“进程盒子”，而是一个个轻量的、可被直接调度的“线程执行流”
- 进程蜕变为资源的容器，而线程则是容器中活跃的工作单元

线程 (Thread) 的调度 (Schedule)

★ 线程基本优先级矩阵

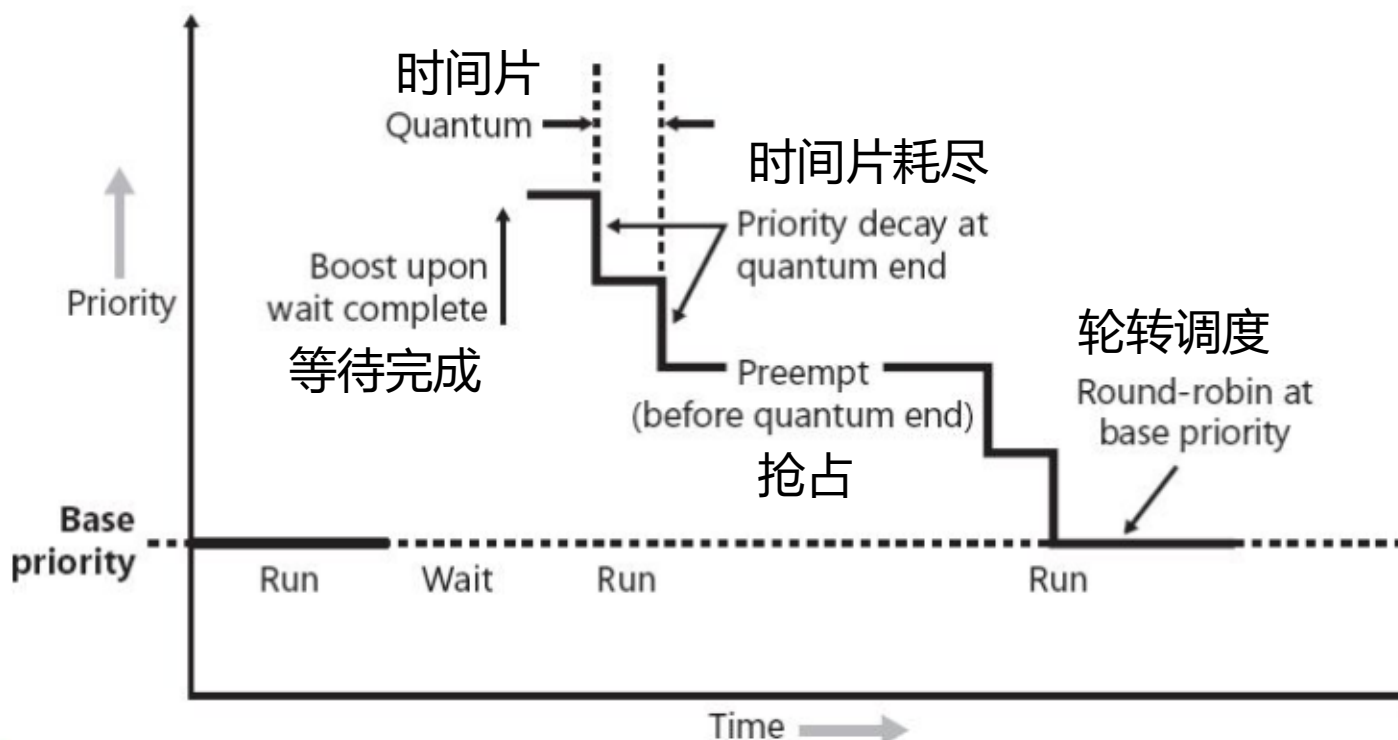
Windows线程的基本优先级

		Win32 Process Classes					
		Realtime	High	Above Normal	Normal	Below Normal	Idle
Win32 Thread Priorities	Time-critical	31	15	15	15	15	15
	Highest	26	15	12	10	8	6
	Above-normal	25	14	11	9	7	5
	Normal	24	13	10	8	6	4
	Below-normal	23	12	9	7	5	3
	Lowest	22	11	8	6	4	2
	Idle	16	1	1	1	1	1

线程 (Thread) 的调度 (Schedule)

★ 动态优先级

每个线程的“动态优先级”以线程的基本优先级为初始值，随着进程所做工作类型的不同而上下浮动。



思考（三选一）

1. 进程管理时的两个重要考虑是公平和效率，除此之外，还有什么因素需要考虑？
2. 各个进程调度算法的特点、适应性如何？
3. 目前讲的进程管理涉及到的硬件是CPU，那GPU呢？